
Batch 24 — Asset Inventory + Maintenance Control Pack

Daniel Macharia <danielmacharia43@gmail.com>
To: <daniel@ics.ac.ke>

Tue, 16 Jun at 08:25

Batch 24 — Asset Inventory + Maintenance Control Pack

Meat Lovers CIMS powered by YohPal

24A.

database/migrations/049_asset_assignments.sql

```
CREATE TABLE asset_assignments (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  
  asset_id BIGINT NOT NULL,  
  assigned_to BIGINT,  
  department ENUM(  
    'STORE',  
    'KITCHEN',  
    'SERVICE',  
    'DISPATCH',  
    'BAR',  
    'HRM',  
    'FINANCE',  
    'ADMIN'  
  ) NOT NULL,  
  
  assigned_date DATE NOT NULL,  
  returned_date DATE NULL,  
  
  assignment_status ENUM(  
    'ASSIGNED',  
    'RETURNED',  
    'LOST',  
    'DAMAGED'  
  ) DEFAULT 'ASSIGNED',  
  
  notes TEXT,  
  
  FOREIGN KEY (asset_id) REFERENCES assets(id),  
  FOREIGN KEY (assigned_to) REFERENCES users(id),  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

24B.

database/migrations/050_asset_maintenance_schedules.sql

```
CREATE TABLE asset_maintenance_schedules (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  
  asset_id BIGINT NOT NULL,  
  
  maintenance_type ENUM(  
    'INSPECTION',  
    'SERVICE',  
    'REPAIR',  
    'CLEANING',  
    'CALIBRATION'  
  ) NOT NULL,  
  
  scheduled_date DATE NOT NULL,  
  
  maintenance_status ENUM(  
    'PENDING',  
    'IN_PROGRESS',  
    'COMPLETED',  
    'OVERDUE',  
    'CANCELLED'
```

```

    ) DEFAULT 'PENDING',

    assigned_to BIGINT,
    notes TEXT,

    FOREIGN KEY (asset_id) REFERENCES assets(id),
    FOREIGN KEY (assigned_to) REFERENCES users(id),

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

```

24C.

database/migrations/051_asset_repair_logs.sql

```

CREATE TABLE asset_repair_logs (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,

    asset_id BIGINT NOT NULL,

    repair_date DATE NOT NULL,
    repair_cost DECIMAL(12,2) DEFAULT 0,

    repair_description TEXT,
    repaired_by VARCHAR(255),

    recorded_by BIGINT,

    FOREIGN KEY (asset_id) REFERENCES assets(id),
    FOREIGN KEY (recorded_by) REFERENCES users(id),

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

```

24D.

database/migrations/052_asset_damage_reports.sql

```

CREATE TABLE asset_damage_reports (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,

    asset_id BIGINT NOT NULL,

    reported_by BIGINT,
    damage_date DATE NOT NULL,

    damage_description TEXT NOT NULL,

    damage_status ENUM(
        'REPORTED',
        'UNDER_REVIEW',
        'REPAIR_REQUIRED',
        'WRITEOFF_REQUESTED',
        'RESOLVED'
    ) DEFAULT 'REPORTED',

    estimated_loss DECIMAL(12,2) DEFAULT 0,

    FOREIGN KEY (asset_id) REFERENCES assets(id),
    FOREIGN KEY (reported_by) REFERENCES users(id),

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

```

24E.

database/migrations/053_asset_writeoff_requests.sql

```

CREATE TABLE asset_writeoff_requests (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,

    asset_id BIGINT NOT NULL,
    requested_by BIGINT NOT NULL,

    request_reason TEXT NOT NULL,
    estimated_loss DECIMAL(12,2) DEFAULT 0,

```

```

writeoff_status ENUM(
    'PENDING',
    'APPROVED',
    'REJECTED',
    'APPLIED'
) DEFAULT 'PENDING',

approved_by BIGINT,
approved_at TIMESTAMP NULL,

FOREIGN KEY (asset_id) REFERENCES assets(id),
FOREIGN KEY (requested_by) REFERENCES users(id),
FOREIGN KEY (approved_by) REFERENCES users(id),

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

```

24F. Update

backend/routes/api.php

```

$router->get('/api/assets/assignments', [AssetsController::class, 'assignments']);
$router->post('/api/assets/assignments', [AssetsController::class, 'createAssignment']);

$router->get('/api/assets/maintenance-schedules', [AssetsController::class, 'maintenanceSchedules']);
$router->post('/api/assets/maintenance-schedules', [AssetsController::class, 'createMaintenanceSchedule']);
$router->post('/api/assets/maintenance-schedules/{id}/complete', [AssetsController::class,
'completeMaintenanceSchedule']);

$router->get('/api/assets/repair-logs', [AssetsController::class, 'repairLogs']);
$router->post('/api/assets/repair-logs', [AssetsController::class, 'createRepairLog']);

$router->get('/api/assets/damage-reports', [AssetsController::class, 'damageReports']);
$router->post('/api/assets/damage-reports', [AssetsController::class, 'createDamageReport']);

$router->get('/api/assets/writeoff-requests', [AssetsController::class, 'writeoffRequests']);
$router->post('/api/assets/writeoff-requests', [AssetsController::class, 'createWriteoffRequest']);
$router->post('/api/assets/writeoff-requests/{id}/approve', [AssetsController::class,
'approveWriteoffRequest']);
$router->post('/api/assets/writeoff-requests/{id}/apply', [AssetsController::class, 'applyWriteoffRequest']);

$router->get('/api/assets/lifecycle-dashboard', [AssetsController::class, 'lifecycleDashboard']);

```

24G. Update

backend/app/Controllers/AssetsController.php

Add these methods inside the class:

```

public function assignments(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT', 'HR']);

    $stmt = Database::connection()->query(
        "SELECT
            aa.*,
            a.asset_name,
            u.full_name AS assigned_to_name
        FROM asset_assignments aa
        JOIN assets a ON a.id = aa.asset_id
        LEFT JOIN users u ON u.id = aa.assigned_to
        ORDER BY aa.created_at DESC"
    );

    Response::success(['asset_assignments' => $stmt->fetchAll()]);
}

public function createAssignment(): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT', 'HR']);
    $data = Request::body();

    $stmt = Database::connection()->prepare(
        "INSERT INTO asset_assignments
        (asset_id, assigned_to, department, assigned_date, notes)
        VALUES (:asset_id, :assigned_to, :department, :assigned_date, :notes)"
    );

    $stmt->execute([
        'asset_id' => $data['asset_id'],
        'assigned_to' => $data['assigned_to'] ?? null,
    ]);
}

```

```

        'department' => $data['department'],
        'assigned_date' => $data['assigned_date'],
        'notes' => $data['notes'] ?? null,
    ]);

    AuditLogger::log((int) $user['id'], 'ASSETS', 'CREATE_ASSET_ASSIGNMENT', (int) $data['asset_id'], null,
    $data);

    Response::success([], 'Asset assignment created');
}

public function maintenanceSchedules(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT']);

    $stmt = Database::connection()->query(
        "SELECT
            ams.*,
            a.asset_name,
            u.full_name AS assigned_to_name
        FROM asset_maintenance_schedules ams
        JOIN assets a ON a.id = ams.asset_id
        LEFT JOIN users u ON u.id = ams.assigned_to
        ORDER BY ams.scheduled_date ASC"
    );

    Response::success(['maintenance_schedules' => $stmt->fetchAll()]);
}

public function createMaintenanceSchedule(): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT']);
    $data = Request::body();

    $stmt = Database::connection()->prepare(
        'INSERT INTO asset_maintenance_schedules
        (asset_id, maintenance_type, scheduled_date, assigned_to, notes)
        VALUES (:asset_id, :maintenance_type, :scheduled_date, :assigned_to, :notes)'
    );

    $stmt->execute([
        'asset_id' => $data['asset_id'],
        'maintenance_type' => $data['maintenance_type'],
        'scheduled_date' => $data['scheduled_date'],
        'assigned_to' => $data['assigned_to'] ?? null,
        'notes' => $data['notes'] ?? null,
    ]);

    AuditLogger::log((int) $user['id'], 'ASSETS', 'CREATE_MAINTENANCE_SCHEDULE', (int) $data['asset_id'], null,
    $data);

    Response::success([], 'Maintenance schedule created');
}

public function completeMaintenanceSchedule(int $id): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT']);

    $stmt = Database::connection()->prepare(
        "UPDATE asset_maintenance_schedules
        SET maintenance_status = 'COMPLETED'
        WHERE id = :id"
    );

    $stmt->execute(['id' => $id]);

    AuditLogger::log((int) $user['id'], 'ASSETS', 'COMPLETE_MAINTENANCE_SCHEDULE', $id);

    Response::success([], 'Maintenance schedule completed');
}

public function repairLogs(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT']);

    $stmt = Database::connection()->query(
        "SELECT
            arl.*,
            a.asset_name,
            u.full_name AS recorded_by_name
        FROM asset_repair_logs arl
        JOIN assets a ON a.id = arl.asset_id
        LEFT JOIN users u ON u.id = arl.recorded_by
        ORDER BY arl.repair_date DESC"
    );

    Response::success(['repair_logs' => $stmt->fetchAll()]);
}

```

```

}

public function createRepairLog(): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT']);
    $data = Request::body();

    $stmt = Database::connection()->prepare(
        'INSERT INTO asset_repair_logs
        (asset_id, repair_date, repair_cost, repair_description, repaired_by, recorded_by)
        VALUES (:asset_id, :repair_date, :repair_cost, :description, :repaired_by, :recorded_by)'
    );

    $stmt->execute([
        'asset_id' => $data['asset_id'],
        'repair_date' => $data['repair_date'],
        'repair_cost' => $data['repair_cost'] ?? 0,
        'description' => $data['repair_description'] ?? null,
        'repaired_by' => $data['repaired_by'] ?? null,
        'recorded_by' => $user['id'],
    ]);

    if ((float) ($data['repair_cost'] ?? 0) > 0) {
        $finance = Database::connection()->prepare(
            'INSERT INTO finance_transactions
            (transaction_type, category, amount, reference_number, notes, created_by)
            VALUES ("EXPENSE", "Repairs and Maintenance", :amount, :reference, :notes, :created_by)'
        );

        $finance->execute([
            'amount' => $data['repair_cost'],
            'reference' => 'ASSET-REPAIR-' . $data['asset_id'],
            'notes' => $data['repair_description'] ?? 'Asset repair cost',
            'created_by' => $user['id'],
        ]);
    }

    AuditLogger::log((int) $user['id'], 'ASSETS', 'CREATE_REPAIR_LOG', (int) $data['asset_id'], null, $data);

    Response::success([], 'Asset repair log created');
}

public function damageReports(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT', 'HR']);

    $stmt = Database::connection()->query(
        "SELECT
        adr.*,
        a.asset_name,
        u.full_name AS reported_by_name
        FROM asset_damage_reports adr
        JOIN assets a ON a.id = adr.asset_id
        LEFT JOIN users u ON u.id = adr.reported_by
        ORDER BY adr.created_at DESC"
    );

    Response::success(['damage_reports' => $stmt->fetchAll()]);
}

public function createDamageReport(): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT', 'HR']);
    $data = Request::body();

    $stmt = Database::connection()->prepare(
        'INSERT INTO asset_damage_reports
        (asset_id, reported_by, damage_date, damage_description, estimated_loss)
        VALUES (:asset_id, :reported_by, :damage_date, :description, :estimated_loss)'
    );

    $stmt->execute([
        'asset_id' => $data['asset_id'],
        'reported_by' => $user['id'],
        'damage_date' => $data['damage_date'],
        'description' => $data['damage_description'],
        'estimated_loss' => $data['estimated_loss'] ?? 0,
    ]);

    $assetUpdate = Database::connection()->prepare(
        "UPDATE assets SET asset_status = 'DAMAGED' WHERE id = :id"
    );

    $assetUpdate->execute(['id' => $data['asset_id']]);

    AuditLogger::log((int) $user['id'], 'ASSETS', 'CREATE_DAMAGE_REPORT', (int) $data['asset_id'], null, $data);
}

```

```

        Response::success([], 'Asset damage report created');
    }

    public function writeoffRequests(): void
    {
        Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT']);

        $stmt = Database::connection()->query(
            "SELECT
                awr.*,
                a.asset_name,
                requester.full_name AS requested_by_name,
                approver.full_name AS approved_by_name
            FROM asset_writeoff_requests awr
            JOIN assets a ON a.id = awr.asset_id
            JOIN users requester ON requester.id = awr.requested_by
            LEFT JOIN users approver ON approver.id = awr.approved_by
            ORDER BY awr.created_at DESC"
        );

        Response::success(['writeoff_requests' => $stmt->fetchAll()]);
    }

    public function createWriteoffRequest(): void
    {
        $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT']);
        $data = Request::body();

        $stmt = Database::connection()->prepare(
            'INSERT INTO asset_writeoff_requests
            (asset_id, requested_by, request_reason, estimated_loss)
            VALUES (:asset_id, :requested_by, :reason, :estimated_loss)'
        );

        $stmt->execute([
            'asset_id' => $data['asset_id'],
            'requested_by' => $user['id'],
            'reason' => $data['request_reason'],
            'estimated_loss' => $data['estimated_loss'] ?? 0,
        ]);

        AuditLogger::log((int) $user['id'], 'ASSETS', 'CREATE_WRITEOFF_REQUEST', (int) $data['asset_id'], null,
            $data);

        Response::success([], 'Asset write-off request created');
    }

    public function approveWriteoffRequest(int $id): void
    {
        $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER']);

        $stmt = Database::connection()->prepare(
            "UPDATE asset_writeoff_requests
            SET writeoff_status = 'APPROVED',
                approved_by = :approved_by,
                approved_at = NOW()
            WHERE id = :id"
        );

        $stmt->execute([
            'approved_by' => $user['id'],
            'id' => $id,
        ]);

        AuditLogger::log((int) $user['id'], 'ASSETS', 'APPROVE_WRITEOFF_REQUEST', $id);

        Response::success([], 'Asset write-off request approved');
    }

    public function applyWriteoffRequest(int $id): void
    {
        $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER']);
        $db = Database::connection();

        $stmt = $db->prepare(
            'SELECT * FROM asset_writeoff_requests WHERE id = :id LIMIT 1'
        );

        $stmt->execute(['id' => $id]);
        $request = $stmt->fetch();

        if (!$request) {
            Response::error('Write-off request not found', 404);
        }

        if ($request['writeoff_status'] !== 'APPROVED') {

```

```

        Response::error('Write-off must be approved before applying', 403);
    }

    $asset = $db->prepare(
        "UPDATE assets SET asset_status = 'DISPOSED' WHERE id = :asset_id"
    );

    $asset->execute(['asset_id' => $request['asset_id']]);

    $update = $db->prepare(
        "UPDATE asset_writeoff_requests
        SET writeoff_status = 'APPLIED'
        WHERE id = :id"
    );

    $update->execute(['id' => $id]);

    AuditLogger::log((int) $user['id'], 'ASSETS', 'APPLY_WRITEOFF_REQUEST', $id, null, $request);

    Response::success([], 'Asset write-off applied');
}

public function lifecycleDashboard(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT']);

    $db = Database::connection();

    $summary = $db->query(
        "SELECT
        COUNT(*) AS total_assets,
        SUM(CASE WHEN asset_status = 'ACTIVE' THEN 1 ELSE 0 END) AS active_assets,
        SUM(CASE WHEN asset_status = 'DAMAGED' THEN 1 ELSE 0 END) AS damaged_assets,
        SUM(CASE WHEN asset_status = 'DISPOSED' THEN 1 ELSE 0 END) AS disposed_assets,
        COALESCE(SUM(purchase_cost), 0) AS total_asset_value
        FROM assets"
    )->fetch();

    $maintenance = $db->query(
        "SELECT COUNT(*) AS pending_maintenance
        FROM asset_maintenance_schedules
        WHERE maintenance_status IN ('PENDING', 'OVERDUE')"
    )->fetch();

    $writeoffs = $db->query(
        "SELECT COUNT(*) AS pending_writeoffs
        FROM asset_writeoff_requests
        WHERE writeoff_status = 'PENDING'"
    )->fetch();

    Response::success([
        'summary' => $summary,
        'pending_maintenance' => (int) $maintenance['pending_maintenance'],
        'pending_writeoffs' => (int) $writeoffs['pending_writeoffs'],
    ]);
}

```

24H. Update

apps/admin-web/src/features/operations/api/operationsApi.js

```

assetAssignments: async () => {
    const { data } = await apiClient.get('/assets/assignments')
    return data.data.asset_assignments
},

createAssetAssignment: async (payload) => {
    const { data } = await apiClient.post('/assets/assignments', payload)
    return data
},

assetMaintenanceSchedules: async () => {
    const { data } = await apiClient.get('/assets/maintenance-schedules')
    return data.data.maintenance_schedules
},

createAssetMaintenanceSchedule: async (payload) => {
    const { data } = await apiClient.post('/assets/maintenance-schedules', payload)
    return data
},

completeAssetMaintenanceSchedule: async (id) => {
    const { data } = await apiClient.post(`/assets/maintenance-schedules/${id}/complete`, {})
    return data
}

```

```

},

assetRepairLogs: async () => {
  const { data } = await apiClient.get('/assets/repair-logs')
  return data.data.repair_logs
},

createAssetRepairLog: async (payload) => {
  const { data } = await apiClient.post('/assets/repair-logs', payload)
  return data
},

assetDamageReports: async () => {
  const { data } = await apiClient.get('/assets/damage-reports')
  return data.data.damage_reports
},

createAssetDamageReport: async (payload) => {
  const { data } = await apiClient.post('/assets/damage-reports', payload)
  return data
},

assetWriteoffRequests: async () => {
  const { data } = await apiClient.get('/assets/writeoff-requests')
  return data.data.writeoff_requests
},

createAssetWriteoffRequest: async (payload) => {
  const { data } = await apiClient.post('/assets/writeoff-requests', payload)
  return data
},

approveAssetWriteoffRequest: async (id) => {
  const { data } = await apiClient.post(`/assets/writeoff-requests/${id}/approve`, {})
  return data
},

applyAssetWriteoffRequest: async (id) => {
  const { data } = await apiClient.post(`/assets/writeoff-requests/${id}/apply`, {})
  return data
},

assetLifecycleDashboard: async () => {
  const { data } = await apiClient.get('/assets/lifecycle-dashboard')
  return data.data
},

```

241.

apps/admin-web/src/features/assets/pages/AssetLifecycleDashboardPage.jsx

```

import React from 'react'
import { useQuery } from '@tanstack/react-query'
import Page from '../../../../components/ui/Page'
import Card from '../../../../components/ui/Card'
import LoadingBlock from '../../../../components/ui/LoadingBlock'
import StatusMessage from '../../../../components/ui/StatusMessage'
import { operationsApi } from '../../../../operations/api/operationsApi'

export default function AssetLifecycleDashboardPage() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['asset-lifecycle-dashboard'],
    queryFn: operationsApi.assetLifecycleDashboard,
  })

  return (
    <Page title="Asset Lifecycle Dashboard" subtitle="Asset value, condition, maintenance, and write-off control">
      {isLoading ? <LoadingBlock label="Loading asset lifecycle dashboard..." /> : null}
      {error ? <StatusMessage error={error.message} /> : null}

      {data ? (
        <div style={{ display: 'grid', gridTemplateColumns: 'repeat(6, minmax(0, 1fr))', gap: 16 }}>
          <Card title="Total Assets"><strong>{data.summary.total_assets}</strong></Card>
          <Card title="Active"><strong>{data.summary.active_assets}</strong></Card>
          <Card title="Damaged"><strong>{data.summary.damaged_assets}</strong></Card>
          <Card title="Disposed"><strong>{data.summary.disposed_assets}</strong></Card>
          <Card title="Asset Value"><strong>{data.summary.total_asset_value}</strong></Card>
          <Card title="Pending Maintenance"><strong>{data.pending_maintenance}</strong></Card>
          <Card title="Pending Write-Offs"><strong>{data.pending_writeoffs}</strong></Card>
        </div>
      ) : null}
    </Page>
  )
}

```



```
}
```

24J.

apps/admin-web/src/features/assets/pages/AssetAssignmentsPage.jsx

```
import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../../operations/api/operationsApi'

export default function AssetAssignmentsPage() {
  return (
    <OperationalFormPage
      title="Asset Assignments"
      subtitle="Assign assets to staff or departments"
      queryKey="asset-assignments"
      queryFn={operationsApi.assetAssignments}
      mutationFn={operationsApi.createAssetAssignment}
      initialForm={{
        asset_id: '',
        assigned_to: '',
        department: 'KITCHEN',
        assigned_date: '',
        notes: '',
      }}
      buildPayload={(form) => ({
        asset_id: Number(form.asset_id),
        assigned_to: form.assigned_to ? Number(form.assigned_to) : null,
        department: form.department,
        assigned_date: form.assigned_date,
        notes: form.notes,
      })}
      fields={[
        { name: 'asset_id', label: 'Asset ID', type: 'number' },
        { name: 'assigned_to', label: 'Assigned Staff ID optional', type: 'number' },
        {
          name: 'department',
          label: 'Department',
          type: 'select',
          options: [
            { value: 'STORE', label: 'Store' },
            { value: 'KITCHEN', label: 'Kitchen' },
            { value: 'SERVICE', label: 'Service' },
            { value: 'DISPATCH', label: 'Dispatch' },
            { value: 'BAR', label: 'Bar' },
            { value: 'HRM', label: 'HRM' },
            { value: 'FINANCE', label: 'Finance' },
            { value: 'ADMIN', label: 'Admin' },
          ],
        },
        { name: 'assigned_date', label: 'Assigned Date', type: 'date' },
        { name: 'notes', label: 'Notes', type: 'textarea' },
      ]}
      columns={[
        { key: 'asset_name', label: 'Asset' },
        { key: 'assigned_to_name', label: 'Assigned To' },
        { key: 'department', label: 'Department' },
        { key: 'assigned_date', label: 'Assigned Date' },
        { key: 'assignment_status', label: 'Status' },
      ]}
      submitLabel="Assign Asset"
    />
  )
}
```

24K.

apps/admin-web/src/features/assets/pages/AssetMaintenancePage.jsx

```
import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../../operations/api/operationsApi'

export default function AssetMaintenancePage() {
  return (
    <OperationalFormPage
      title="Asset Maintenance"
      subtitle="Schedule inspections, service, repairs, and cleaning"
      queryKey="asset-maintenance"
      queryFn={operationsApi.assetMaintenanceSchedules}
      mutationFn={operationsApi.createAssetMaintenanceSchedule}
      initialForm={{

```

```

        asset_id: '',
        maintenance_type: 'INSPECTION',
        scheduled_date: '',
        assigned_to: '',
        notes: '',
    })
    buildPayload={({form}) => ({
        asset_id: Number(form.asset_id),
        maintenance_type: form.maintenance_type,
        scheduled_date: form.scheduled_date,
        assigned_to: form.assigned_to ? Number(form.assigned_to) : null,
        notes: form.notes,
    })}
    fields=[
        { name: 'asset_id', label: 'Asset ID', type: 'number' },
        {
            name: 'maintenance_type',
            label: 'Maintenance Type',
            type: 'select',
            options: [
                { value: 'INSPECTION', label: 'Inspection' },
                { value: 'SERVICE', label: 'Service' },
                { value: 'REPAIR', label: 'Repair' },
                { value: 'CLEANING', label: 'Cleaning' },
                { value: 'CALIBRATION', label: 'Calibration' },
            ],
        },
        { name: 'scheduled_date', label: 'Scheduled Date', type: 'date' },
        { name: 'assigned_to', label: 'Assigned Staff ID optional', type: 'number' },
        { name: 'notes', label: 'Notes', type: 'textarea' },
    ]
    columns=[
        { key: 'asset_name', label: 'Asset' },
        { key: 'maintenance_type', label: 'Type' },
        { key: 'scheduled_date', label: 'Date' },
        { key: 'maintenance_status', label: 'Status' },
        { key: 'assigned_to_name', label: 'Assigned To' },
    ]
    submitLabel="Schedule Maintenance"
    />
)
}

```

24L.

apps/admin-web/src/features/assets/pages/AssetRepairLogsPage.jsx

```

import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../../operations/api/operationsApi'

export default function AssetRepairLogsPage() {
    return (
        <OperationalFormPage
            title="Asset Repair Logs"
            subtitle="Record repairs and maintenance costs"
            queryKey="asset-repair-logs"
            queryFn={operationsApi.assetRepairLogs}
            mutationFn={operationsApi.createAssetRepairLog}
            initialForm={{
                asset_id: '',
                repair_date: '',
                repair_cost: '',
                repair_description: '',
                repaired_by: '',
            }}
            buildPayload={({form}) => ({
                asset_id: Number(form.asset_id),
                repair_date: form.repair_date,
                repair_cost: Number(form.repair_cost || 0),
                repair_description: form.repair_description,
                repaired_by: form.repaired_by,
            })}
            fields=[
                { name: 'asset_id', label: 'Asset ID', type: 'number' },
                { name: 'repair_date', label: 'Repair Date', type: 'date' },
                { name: 'repair_cost', label: 'Repair Cost', type: 'number' },
                { name: 'repair_description', label: 'Repair Description', type: 'textarea' },
                { name: 'repaired_by', label: 'Repaired By' },
            ]
            columns=[
                { key: 'asset_name', label: 'Asset' },
                { key: 'repair_date', label: 'Repair Date' },
                { key: 'repair_cost', label: 'Cost' },
            ]
        />
    )
}

```

```

    { key: 'repair_description', label: 'Description' },
    { key: 'repaired_by', label: 'Repaired By' },
    { key: 'recorded_by_name', label: 'Recorded By' },
  ]}
  submitLabel="Record Repair"
</>
)
}

```

24M.

apps/admin-web/src/features/assets/pages/AssetDamageReportsPage.jsx

```

import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../../operations/api/operationsApi'

export default function AssetDamageReportsPage() {
  return (
    <OperationalFormPage
      title="Asset Damage Reports"
      subtitle="Report damaged assets and estimated loss"
      queryKey="asset-damage-reports"
      queryFn={operationsApi.assetDamageReports}
      mutationFn={operationsApi.createAssetDamageReport}
      initialForm={{
        asset_id: '',
        damage_date: '',
        damage_description: '',
        estimated_loss: '',
      }}
      buildPayload={(form) => ({
        asset_id: Number(form.asset_id),
        damage_date: form.damage_date,
        damage_description: form.damage_description,
        estimated_loss: Number(form.estimated_loss || 0),
      })}
      fields={[
        { name: 'asset_id', label: 'Asset ID', type: 'number' },
        { name: 'damage_date', label: 'Damage Date', type: 'date' },
        { name: 'damage_description', label: 'Damage Description', type: 'textarea' },
        { name: 'estimated_loss', label: 'Estimated Loss', type: 'number' },
      ]}
      columns={[
        { key: 'asset_name', label: 'Asset' },
        { key: 'reported_by_name', label: 'Reported By' },
        { key: 'damage_date', label: 'Date' },
        { key: 'damage_status', label: 'Status' },
        { key: 'estimated_loss', label: 'Estimated Loss' },
      ]}
      submitLabel="Report Damage"
    </>
  )
}

```

24N.

apps/admin-web/src/features/assets/pages/AssetWriteoffPage.jsx

```

import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../../operations/api/operationsApi'

export default function AssetWriteoffPage() {
  return (
    <OperationalFormPage
      title="Asset Write-Off Requests"
      subtitle="Request asset disposal/write-off for management approval"
      queryKey="asset-writeoff-requests"
      queryFn={operationsApi.assetWriteoffRequests}
      mutationFn={operationsApi.createAssetWriteoffRequest}
      initialForm={{
        asset_id: '',
        request_reason: '',
        estimated_loss: '',
      }}
      buildPayload={(form) => ({
        asset_id: Number(form.asset_id),
        request_reason: form.request_reason,
        estimated_loss: Number(form.estimated_loss || 0),
      })}
      fields={[

```

```

    { name: 'asset_id', label: 'Asset ID', type: 'number' },
    { name: 'request_reason', label: 'Write-Off Reason', type: 'textarea' },
    { name: 'estimated_loss', label: 'Estimated Loss', type: 'number' },
  ]}
  columns=[
    { key: 'asset_name', label: 'Asset' },
    { key: 'requested_by_name', label: 'Requested By' },
    { key: 'request_reason', label: 'Reason' },
    { key: 'estimated_loss', label: 'Loss' },
    { key: 'writeoff_status', label: 'Status' },
    { key: 'approved_by_name', label: 'Approved By' },
  ]}
  submitLabel="Request Write-Off"
/>
)
}

```

240. Update

apps/admin-web/src/app/routes.jsx

```

import AssetLifecycleDashboardPage from '../features/assets/pages/AssetLifecycleDashboardPage'
import AssetAssignmentsPage from '../features/assets/pages/AssetAssignmentsPage'
import AssetMaintenancePage from '../features/assets/pages/AssetMaintenancePage'
import AssetRepairLogsPage from '../features/assets/pages/AssetRepairLogsPage'
import AssetDamageReportsPage from '../features/assets/pages/AssetDamageReportsPage'
import AssetWriteoffPage from '../features/assets/pages/AssetWriteoffPage'

<Route path="/assets/lifecycle-dashboard" element={<AssetLifecycleDashboardPage />} />
<Route path="/assets/assignments" element={<AssetAssignmentsPage />} />
<Route path="/assets/maintenance" element={<AssetMaintenancePage />} />
<Route path="/assets/repair-logs" element={<AssetRepairLogsPage />} />
<Route path="/assets/damage-reports" element={<AssetDamageReportsPage />} />
<Route path="/assets/writeoff" element={<AssetWriteoffPage />} />

```

24P. Update

apps/admin-web/src/components/common/SidebarNav.jsx

```

['Asset Lifecycle', '/assets/lifecycle-dashboard'],
['Asset Assignments', '/assets/assignments'],
['Asset Maintenance', '/assets/maintenance'],
['Asset Repair Logs', '/assets/repair-logs'],
['Asset Damage Reports', '/assets/damage-reports'],
['Asset Write-Offs', '/assets/writeoff'],

```

24Q.

docs/ASSET_INVENTORY_MAINTENANCE_CONTROL_PACK.md

Batch 24 – Asset Inventory + Maintenance Control Pack

System

Meat Lovers CIMS powered by YohPal

Purpose

This batch turns the asset register into a controlled asset lifecycle system.

Features Added

1. Asset assignment
2. Maintenance schedules
3. Repair logs
4. Damage reports
5. Asset write-off approval placeholder
6. Asset lifecycle dashboard

Asset Assignment

Assets can be assigned to:

- staff
- departments

Departments include:

- store
- kitchen

- service
- dispatch
- bar
- HRM
- finance
- admin

Maintenance Schedules

Maintenance can be scheduled as:

- inspection
- service
- repair
- cleaning
- calibration

Repair Logs

Repair logs record:

- asset
- repair date
- repair cost
- repair description
- repaired by

Repair cost also creates a finance expense under:

```text

Repairs and Maintenance

## Damage Reports

Damage reports record:

- damaged asset
- damage date
- damage description
- estimated loss
- reporting user

Damaged assets are automatically marked as:

DAMAGED

## Write-Off Control

Write-off requests can be:

- pending
- approved
- rejected
- applied

When applied, the asset status becomes:

DISPOSED

## Lifecycle Dashboard

Dashboard shows:

- total assets
- active assets
- damaged assets
- disposed assets
- total asset value
- pending maintenance
- pending write-offs

## Business Value

This helps Meat Lovers:

- protect equipment
- assign responsibility
- reduce asset loss
- track repair costs

- prevent unapproved disposal
- improve maintenance planning

---

#### # Batch 24 Outcome

Asset inventory + maintenance now includes:

- ✅ asset assignment
- ✅ maintenance schedules
- ✅ repair logs
- ✅ damage reports
- ✅ asset write-off approval placeholder
- ✅ asset lifecycle dashboard

#### # Next Smart Move

Build \*\*Batch 25 – HRM Attendance + Shift Control Pack\*\*, including staff shifts, attendance clock-in/out, lateness tracking, absence tracking, duty roster, payroll placeholder, and HRM compliance dashboard.